ТЕМА 2. ОСНОВНЫЕ ЭЛЕМЕНТЫ HTML

ЛЕКЦИЯ 2.33. ВАЛИДАЦИЯ ФОРМЫ

Поле для ввода. Атрибут required. Атрибут pattern. Ограничение длины и допустимых значений.

Перед отправкой данных на сервер важно убедиться, что поля формы заполнены в корректном формате. Это называется валидацией на стороне клиента и помогает убедиться, что данные, введённые в каждый элемент формы, соответствуют предъявленным к ним требованиям. Это позволит оградить сервер от получения как откровенно глупых данных, так и от враждебных.

Валидация на стороне клиента — это первичная проверка введённых пользователем данных, которая существенно улучшает удобство взаимодействия с интерфейсом приложения.

Обнаружение некорректных данных на стороне клиента позволяет пользователю немедленно их исправить. Если проверка происходит только на стороне сервера, процесс заполнения может быть более трудоёмким, так как требует многократной пересылки от клиента к серверу и обратно отправляемых данных и сообщений о возможных ошибках.

Язык HTML5 кроме широких возможностей по выстраиванию семантической структуры страницы обладает функционалом по первичной проверке данных введенных пользователем.

Проверка осуществляется путем добавления особых атрибутов к элементам формы.

Но, не следует рассматривать валидацию на стороне клиента как достаточную меру безопасности! Любые данные, отправляемые через форму, необходимо дополнительно проверять на безопасность и на стороне сервера, поскольку валидацию на стороне клиента достаточно просто обойти.

Разговор о валидации продолжится в соответствующих темах курсов по JavaScript и PHP.

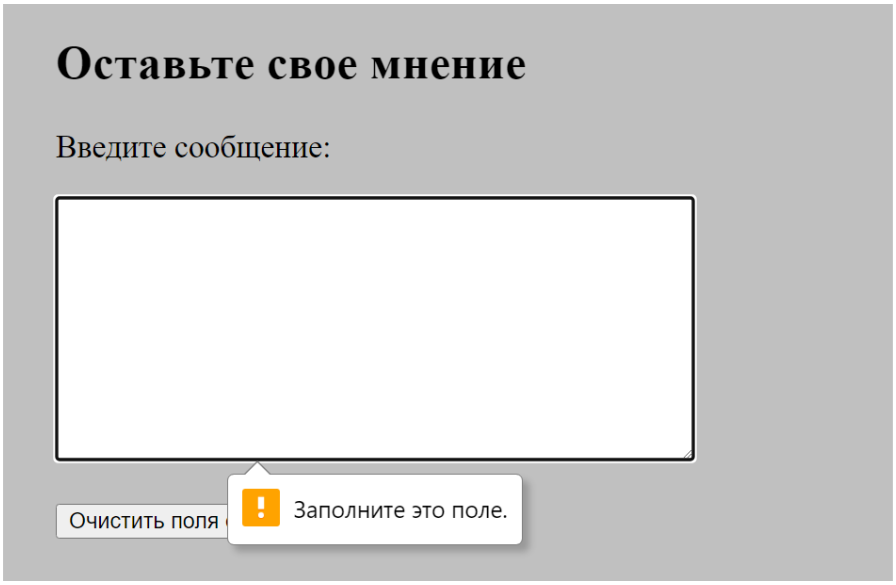
©Pechora-proger

Атрибут required

Атрибут устанавливает поле формы обязательным для заполнения перед отправкой формы на сервер. Если обязательное поле пустое, браузер выведет сообщение об ошибке, а форма отправлена не будет.

Его нельзя использовать для скрытых элементов (атрибут **type=hidden**) изображений или кнопки (**submit**, **reset** или **button**).

Наличие такого атрибута поможет получить от пользователя именно те данные, которые вам жизненно необходимы, например адрес электронной почты или телефон, при этом необязательный список увлечений пользователь будет отправлять по своему усмотрению.



Выбор элемента

Но мало добиться от пользователя просто механического заполнения полей. Важно, чтобы их содержимое было достоверным.

Первый способ ограничить набор вводимых данных – правильный выбор значения атрибута **type**. Определяем с его помощью тип данных, на который рассчитано поле: число, email-адрес или какой-то другой из предустановленных.

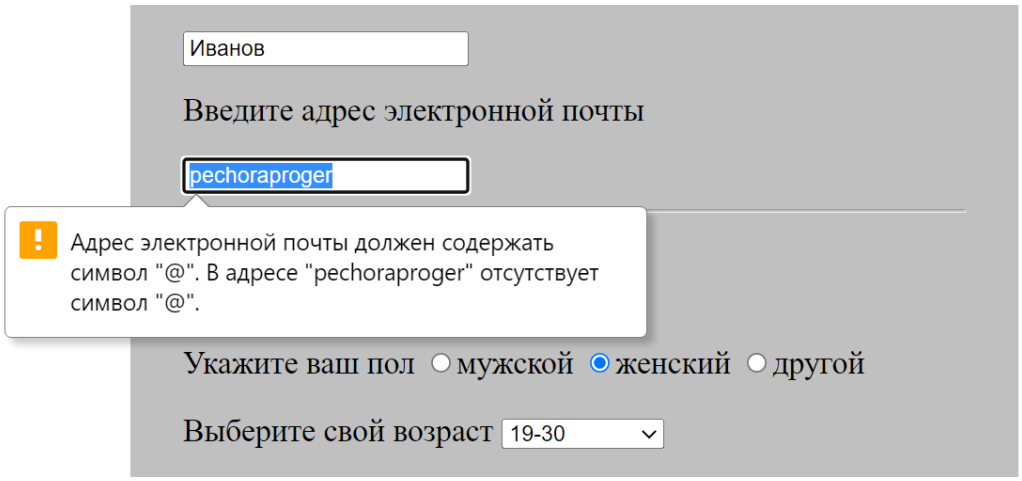
Мы уже затрагивали этот вопрос, но сейчас рассмотрим его подробнее.

input type="email" - используются, чтобы позволить пользователю вводить и редактировать адрес электронной почты.

©Pechora-proger

В **input** содержимое атрибута **value** автоматически проверяется на соответствие **синтаксиса** электронной почты. В частности, есть три возможных формата значений, которые пройдут проверку:

- Пустая строка (""), указывающая, что пользователь не вводил значение или что значение было удалено.
- Единый правильно сформированный адрес электронной почты. Это не обязательно означает, что адрес электронной почты существует, но он, по крайней мере, отформатирован правильно. Проще говоря, это означает username@domain или username@domain.tld.
- Если и только если указан атрибут **multiple**, значение может быть списком правильно сформированных адресов электронной почты, разделенных запятыми. Любые завершающие и ведущие пробелы удаляются из каждого адреса в списке.



Важно! Браузер не проверяет правильность введенного имени домена или существует ли такой адрес на самом деле, не организует никакой связи с введенным адресом (например, отправка письма).

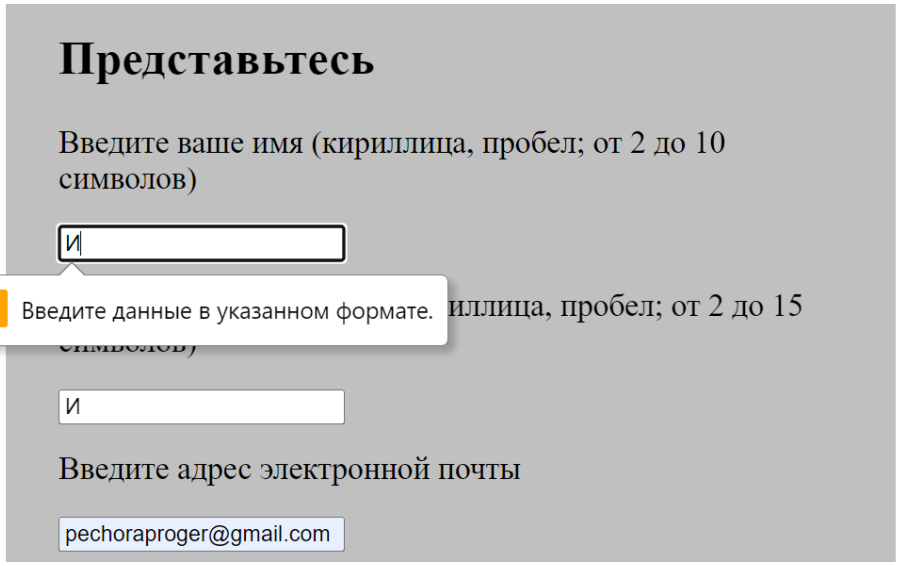
Осуществляется механическая проверка на соответствие шаблону **[какие-то символы]@[какой-то символ]**. Такие шаблоны можно составлять самостоятельно и применять их к разным элементам формы. Для их записи используют регулярные выражения, а указания браузеру на их использование дает атрибут **pattern**.

Атрибут pattern

©Pechora-proger

Итак, **pattern="выражение"** - определяет регулярное выражение, которому должно соответствовать значение элемента формы. Является атрибутом для полей ввода с типом **text**, **tel**, **email**, **url**, **password** и **search**.

Если присутствует атрибут **pattern**, то форма не будет отправляться, пока поле не будет заполнено правильно.



Примеры некоторых регулярных выражений представлены ниже

[abc] —символ a, за которым следует b, за которым следует c.

[ab?c] —символ a, за которым может следовать b, за которым следует c. (ac или abc)

[ab*c] —символ a, за которым может следовать любое количество символов b, за которыми следует c. (ac , abc, abbbbbc, и так далее).

[a|b] —символ a или b.

[abc|xyz] —abc или в точности xyz (но не abcxyz или a или y, и так далее).

[A-Za-z] Только латинская буква в любом регистре.

[А-Яа-яЁё] Только русская буква в любом регистре.

[0-9]{,3} Не более трёх цифр.

[0-9]{5,10} От пяти до десяти цифр.

\s Пробел.

©Pechora-proger

[а-яА-ЯёЁ\-\s]{2,10} Русские буквы в любом регистре, пробел, от двух до десяти символов.

Регулярные выражения широко применяются не только при проверке данных в html-формах с помощью pattern-ов, но и в программировании, в том числе на Javascript и PHP (см. планы курсов в меню группы).

Атрибуты min/max

В полях, предназначенных для ввода чисел, диапазон допустимых значений можно определить с помощью атрибутов **min** и **max**. Если поле содержит значение за его пределами, оно будет не валидным.

Атрибут step

Поля для ввода **number**, **range** или **date** могут иметь атрибут **step**, который задаёт шаг увеличения или уменьшения значения при использовании кнопок вверх и вниз. По умолчанию это значение равно 1. Значит, дробные числа в поле являться валидными не будут.

Ограничение длины вводимых значений

Можно ограничить максимально допустимое количество символов для текстовых полей с помощью атрибутов **minlength** и **maxlength**. Поле будет не валидным, если количество символов его содержимого будет меньше **minlength** или больше **maxlength**.

Важно! В приложенном файле валидации реализован пример формы обратной связи. Изучите поведение формы с текущими и своими собственными настройками.

©Pechora-proger